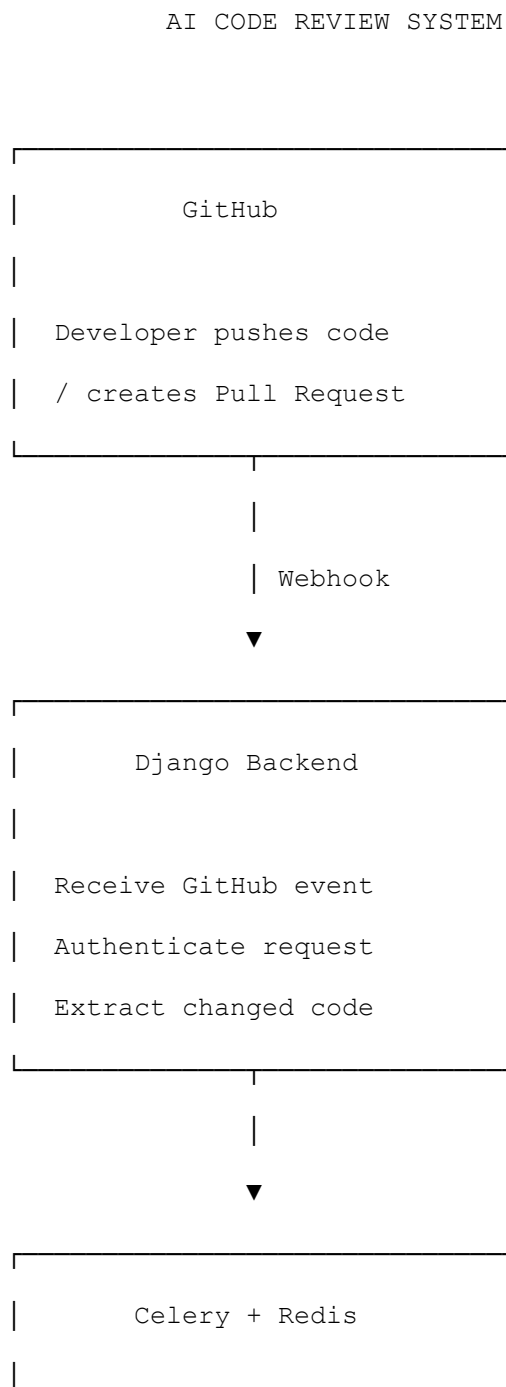
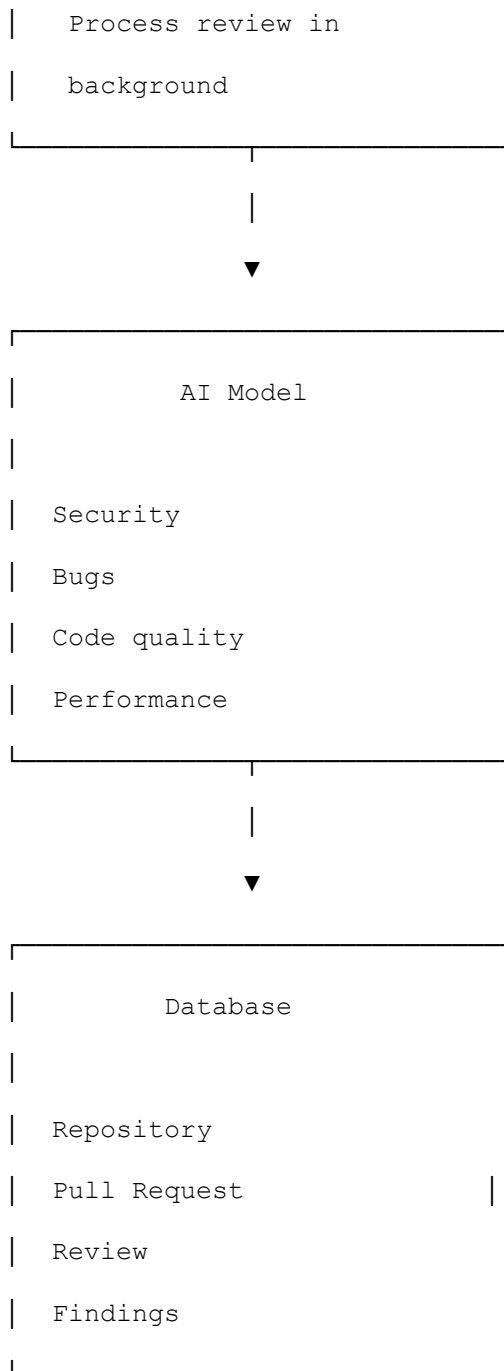


Yes. Since you're building your **AI Code Review Assistant with Django**, starting with the **backend first** is a good approach. The backend is where the main logic happens: receiving GitHub code, sending it to the AI, storing the review, and returning the results.

1. First understand what a backend does

Think of your project like this:





So before worrying about React or the frontend, you need to make this backend pipeline work.

2. What you need to know

I recommend learning these **eight areas**, in this order.

Area	Why you need it
Python	Django is written in Python
Django	Builds your backend
Django REST Framework	Creates APIs for your frontend/GitHub
Databases & SQL	Stores users, repositories, reviews, findings
Git/GitHub	Connects your system to repositories
Webhooks	Allows GitHub to notify your backend
APIs	Allows Django to communicate with GitHub and AI
Celery + Redis	Handles long AI reviews in the background
You don't need to master everything before starting. Learn each part while building.	

3. Python

You should be comfortable with:

variables

if/else

loops

functions

classes

lists

dictionaries

exceptions

modules

virtual environments

pip

For example:

```
def calculate_score(findings):
    return len(findings) * 10
```

You should understand what this does before moving deeply into Django.

Why?

Django applications are Python applications.

4. Django

You need to understand Django's basic architecture.

A simplified structure is:

Django Project

|

|— settings.py

|— urls.py

|

|— review/

|— models.py

|— views.py

|— urls.py

|— serializers.py

|— admin.py

The important files are:

models.py

Defines your database structure.

For example:

```
class Review(models.Model):  
    repository = models.CharField(max_length=255)  
    pull_request = models.IntegerField()  
    status = models.CharField(max_length=50)  
    created_at = models.DateTimeField(auto_now_add=True)
```

This represents information that will be stored in your database.

views.py

Contains backend logic.

For example:

```
def review_code(request):  
    # receive code  
  
    # send code to AI  
  
    # save result  
  
    return JsonResponse({"status": "completed"})
```

urls.py

Defines your API endpoints.

For example:

```
urlpatterns = [  
    path("review/", review_code),  
]
```

Then your application could have:

```
POST /api/review/
```

5. Django REST Framework

For your project, I strongly recommend **Django REST Framework (DRF)**.

Instead of your frontend communicating directly with Django pages, it communicates through APIs.

For example:

```
Frontend
```

|

| POST /api/reviews/



Django REST API

|



Database

You might eventually have:

POST /api/repositories/

GET /api/repositories/

GET /api/reviews/

GET /api/reviews/123/

POST /api/reviews/

The frontend could then display:

Review #123

Security: 82%

Code Quality: 76%

Performance: 91%

Issues:

1. SQL injection vulnerability
2. Missing input validation
3. Inefficient database query

6. Databases

You need to understand basic SQL/database concepts.

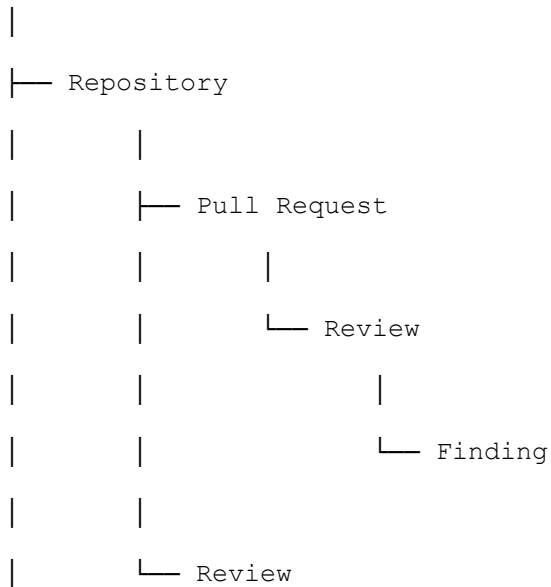
For your project, I would eventually use:

PostgreSQL

During early development, SQLite is fine.

Your database might look like:

User



For example:

Repository

id

name

github_url

owner

created_at

Pull Request

id
repository
github_pr_number
title
status

Review

id
pull_request
score
summary
status
created_at

Finding

id
review
file
line
severity
category
description
suggestion

This is extremely important because your AI's results need to be stored.

7. Git and GitHub

You need to understand:

`git clone`

`git status`

`git add`

`git commit`

`git push`

`git pull`

`git branch`

`git checkout`

`git merge`

And understand what a:

- Repository
- Commit
- Branch
- Pull Request
- Webhook
- GitHub API

is.

For your system, the important workflow is:

Developer

|

| `git push`

▼

GitHub

|

| `webhook`



Django

8. GitHub Webhooks

This is one of the **most important things for your project**.

A webhook allows GitHub to automatically notify your Django server.

For example:

Developer pushes code



GitHub detects push



GitHub sends POST request



`https://yourserver.com/api/github/webhook/`



Django receives event

The Django backend might receive information like:

```
{
  "repository": {
    "name": "my-project"
  },
  "pull_request": {
    "number": 15
  },
  "action": "opened"
}
```

Your backend then says:

A new Pull Request has arrived. Start a code review.

9. GitHub API

The webhook tells your backend **that something happened**.

The GitHub API allows your backend to **get information from GitHub**.

For example:

GitHub webhook

↓

"PR #15 was opened"

↓

Django

↓

GitHub API

↓

Get changed files

↓

Get source code

↓

Send code to AI

This is where your GitHub token or, preferably for a production system, a GitHub App comes in.

10. AI API

Your backend will also communicate with an AI provider.

For example:

Django

|

| code

▼

AI API

|

| analysis

▼

Django

The AI could be instructed to return structured information such as:

```
{  
  "severity": "high",  
  "category": "security",  
  "file": "views.py",  
  "line": 42,  
  "problem": "Potential SQL injection",  
  "suggestion": "Use parameterized queries"  
}
```

Your backend then saves this information.

11. Celery and Redis

This becomes important because AI code review can take time.

You don't want:

GitHub

↓

Django

↓
AI
↓
20 seconds later
↓
Response

Instead:

GitHub
↓
Django
↓
"Review received"
↓
Celery Queue
↓
Worker
↓
AI
↓
Database

Redis acts as the message broker.

Celery performs the background task.

For example:

```
@shared_task
def review_pull_request(pr_id):
    # get PR
```

```
# get changed files

# send code to AI

# save findings
```

This allows your API to remain responsive.

12. Authentication

Eventually you need to understand authentication.

Your system might have:

User

|

├─ Login

├─ Logout

└─ Dashboard

You need to learn concepts such as:

- Sessions
- Tokens
- JWT
- Password hashing
- Permissions
- Authorization

For example:

User A

↓

Repository A

↓

Reviews A

User A shouldn't be able to access User B's private repositories or reviews.

13. Security

Because your application processes **source code**, security is very important.

You need to understand:

API security

Authentication

Authorization

Rate limiting

Input validation

Django security

Understand:

CSRF

CORS

SQL injection

XSS

Authentication

Secrets management

Never do this:

```
GITHUB_TOKEN = "github_pat_XXXXX"
```

Instead:

```
.env
```

↓

```
Environment variable
```

↓

Django

And add `.env` to:

`.gitignore`

14. API design

Before coding heavily, design your endpoints.

For example:

`/api/auth/login/`

`/api/auth/register/`

`/api/repositories/`

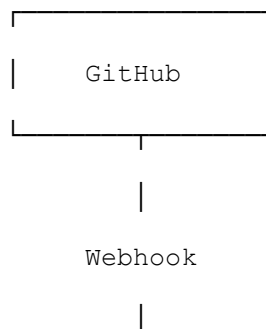
`/api/repositories/{id}/`

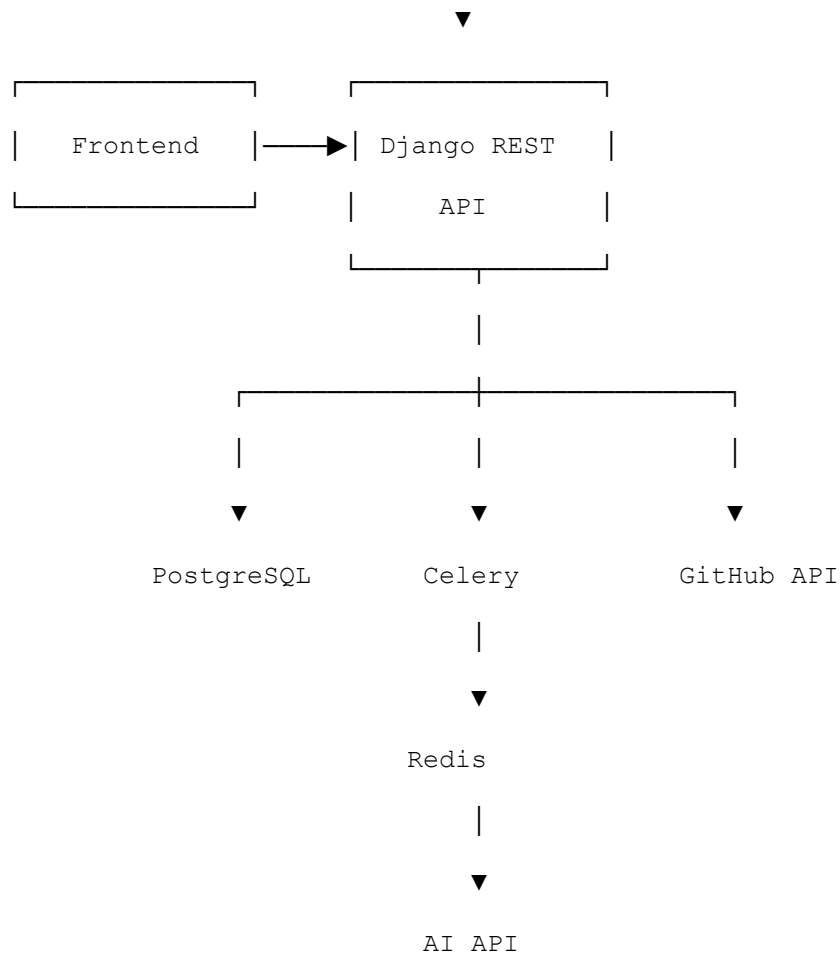
`/api/reviews/`

`/api/reviews/{id}/`

`/api/github/webhook/`

Your architecture could eventually look like:





15. What I recommend you build first

Don't start with the AI.

Build the backend in stages.

Stage 1 — Django

Get this working:

Django

↓

Database

Create your models:

User

Repository

PullRequest

Review

Finding

Stage 2 — REST API

Create:

GET /api/repositories/

POST /api/repositories/

GET /api/reviews/

GET /api/reviews/{id}/

Test everything using **Postman** or `curl`.

Stage 3 — GitHub integration

Connect:

Django

↓

GitHub API

Make Django retrieve:

Repository

Pull Requests

Changed files

Commits

Stage 4 — Webhook

Create:

```
POST /api/github/webhook/
```

Test:

GitHub

↓

Webhook

↓

Django

Stage 5 — Celery + Redis

Move the expensive work into a background task:

Webhook

↓

Celery

↓

Redis

↓

Worker

Stage 6 — AI

Only after the previous stages work:

Code

↓

Celery

↓

AI

↓

Structured review

↓

Database

Stage 7 — GitHub comments

Finally, your system can automatically post something like:

❑ AI Code Review

✖ HIGH: SQL Injection Risk

File: users/views.py

Line: 42

Problem:

User input is directly included in a SQL query.

Suggestion:

Use Django's ORM or parameterized queries.

Severity: HIGH

Category: Security

16. Your backend development roadmap

I would follow this exact order:

BACKEND ROADMAP

1. Python



2. Django



3. PostgreSQL/SQL



4. Django REST API



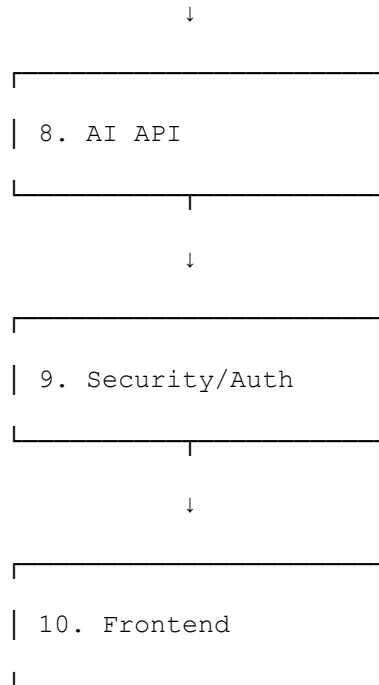
5. Git/GitHub API



6. GitHub Webhooks



7. Celery + Redis



The most important idea

Don't think of the project as **"I need to make an AI website."**

Think of it as:

"I need to build a backend system that receives a GitHub event, retrieves the relevant code, processes it asynchronously, sends it to an AI model, stores the analysis, and optionally posts the results back to GitHub."

Once you understand that workflow, the individual Django components become much easier to understand.

If you're ready to start coding, the **best first step is to create the Django backend and design the database models before connecting GitHub or the AI.**